

Chapter-10 (Recovery & Concurrency Control)

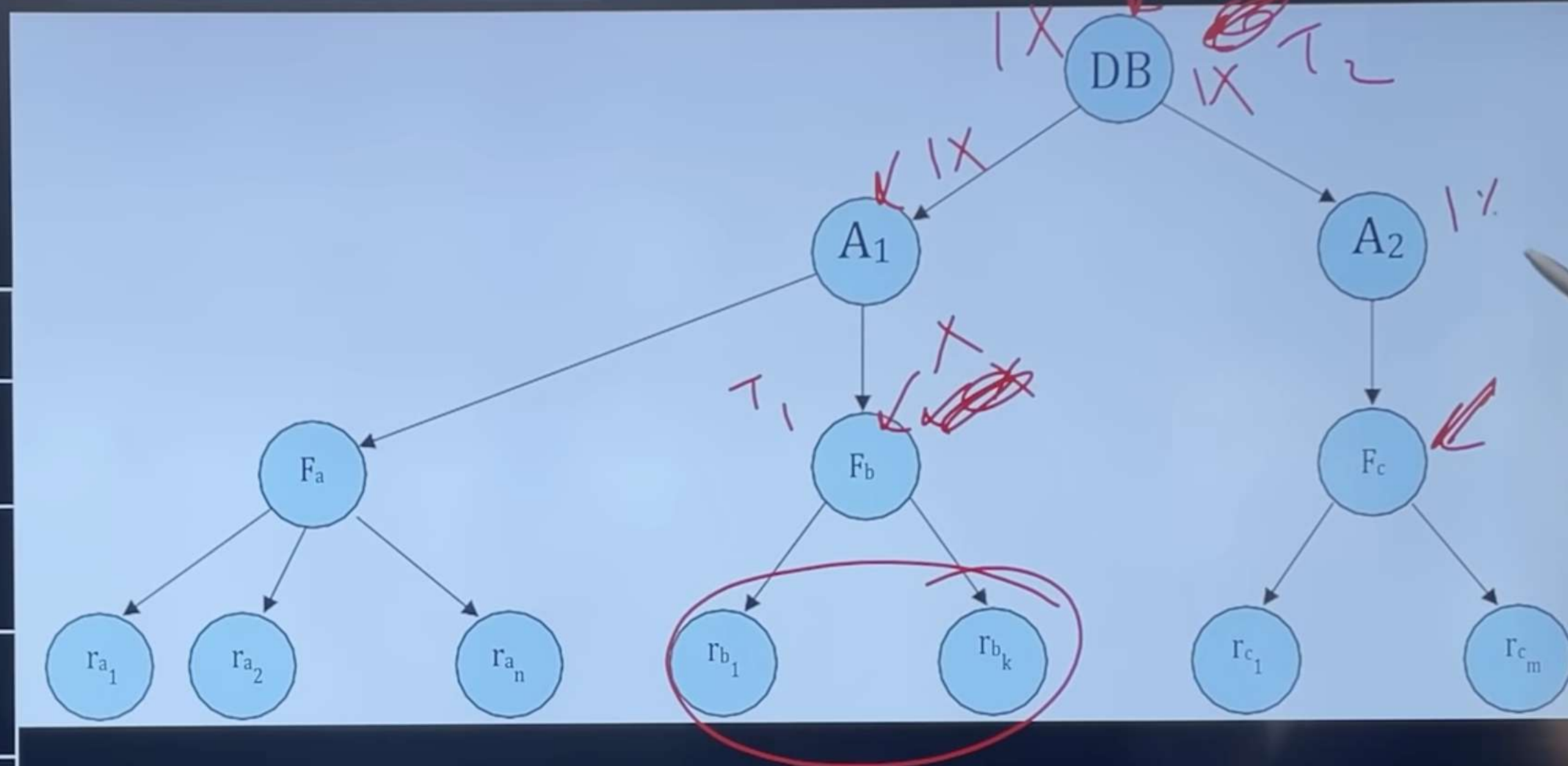
Strict 2pl

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL	YES	YES	YES	YES	NO
Strict 2PL	YES	YES	YES	YES	NO

Multiple Granularity

Multiple Granularity

	IS	IX	<u>S</u>	<u>X</u>
IS	true	true	true	false
IX	true	true	false	false
<u>S</u>	true	false	true	false
<u>X</u>	false	false	false	false



- **Read phase.** During this phase, the system executes transaction T_i . It reads the values of the various data items and stores them in variables local to T_i . It performs all write operations on temporary local variables, without updates of the actual database.
- **Validation phase.** The validation test (described below) is applied to transaction T_i . This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.
- **Write phase.** If the validation test succeeds for transaction T_i , the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

Validation-Based Protocols

- In cases where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low.
- Thus, many of these transactions, if executed without the supervision of a concurrency-control scheme, would nevertheless leave the system in a consistent state.
- A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alternative scheme that imposes less overhead.
- A difficulty in reducing the overhead is that we do not know in advance which transactions will be involved in a conflict. To gain that knowledge, we need a scheme for *monitoring* the system.
- The **validation protocol** requires that each transaction T_i executes in two or three phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order:

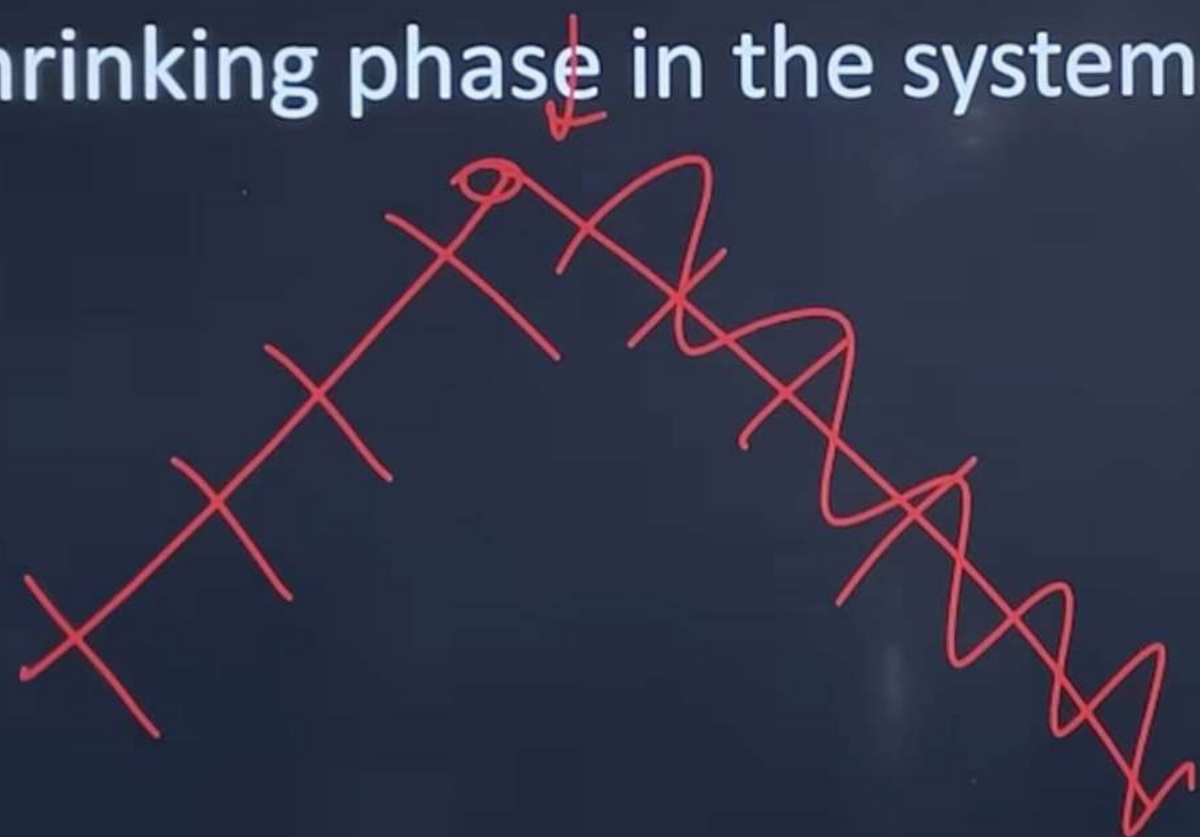
STRICT 2PL

- that all exclusive-mode locks taken by a transaction be held until that transaction commits. This requirement ensures that any data written by an uncommitted transaction are locked in exclusive mode until the transaction commits, preventing any other transaction from reading the data.
- This protocol requires that locking be two phase and also that exclusive –mode locks taken by transaction be held until that transaction commits.
- So it is simplified form of rigorous 2pl



RIGOROUS 2PL

- Requires that all locks be held until the transaction commits. ✓
- This protocol requires that locking be two phase and also all the locks taken by transaction until that transaction commit.
- Hence there is no shrinking phase in the system.



Conservative 2PL

- The idea is there is no growing phase transaction start directly from lock point, i.e. transaction must first acquire all the required locks then only it can start execution. If all the locks are not available then transaction must release the acquired locks and must wait.
- Shrinking phase will work as usual, and transaction can unlock any lock anytime.
- we must have a knowledge in future to understand what is data required by transaction that we can use it

Conservative 2PL (Data Base Management System 4/e, 2nd ed.)

Chapter 10: Recovery & Concurrency Control

2PL ensures conflict serializability and the ordering of transaction over lock points to build a serializable order of a schedule in 2PL.

- If a schedule is allowed in 2PL, protocol then definitely it is always conflict serializable. But it is not necessary that if a schedule is conflict serializable then it will be generated by 2PL.
- Equivalent serial schedule is based on the order of lock points.
- View serializability is also guaranteed.
- Does not ensure freedom from deadlock.
- May cause non-recoverability.
- Cascading rollback may occur.

Properties

- 2PL ensures conflict serializability, and the ordering of transaction over lock points is itself a serializability order of a schedule in 2PL.
- If a schedule is allowed in 2PL protocol then definitely it is always conflict serializable. But it is not necessary that if a schedule is conflict serializable then it will be generated by 2pl.
- Equivalent serial schedule is based on the order of lock points.
- View serializability is also guaranteed.
- Does not ensure freedom from deadlock
- May cause non-recoverability.
- Cascading rollback may occur.

- Initially a transaction is in growing phase and acquires lock as needed and in between can perform operation reach to lock point and once a transaction releases a lock, it can issue no more lock requests i.e. it enters the shrinking phase.

T ₁	T ₂
LOCK-X(A)	
READ(A)	
WRITE(A)	
	LOCK-S(B)
	READ(B)
LOCK-X(B)	
READ(B)	
WRITE(B)	
	LOCK-S(A)
	READ(A)
	UNLOCK(B)
UNLOCK(A)	
UNLOCK(B)	
	UNLOCK(A)

Two phase locking protocol(2PL)

- The protocol ensures that each transaction issue lock and unlock requests in two phases, note that each transaction will be 2 phased not schedule.

Growing phase- A transaction may obtain locks, but not release any locks.

Shrinking phase- A transaction may release locks, but may not obtain any new locks.